

CSE327

SOFTWARE ENGINEERING

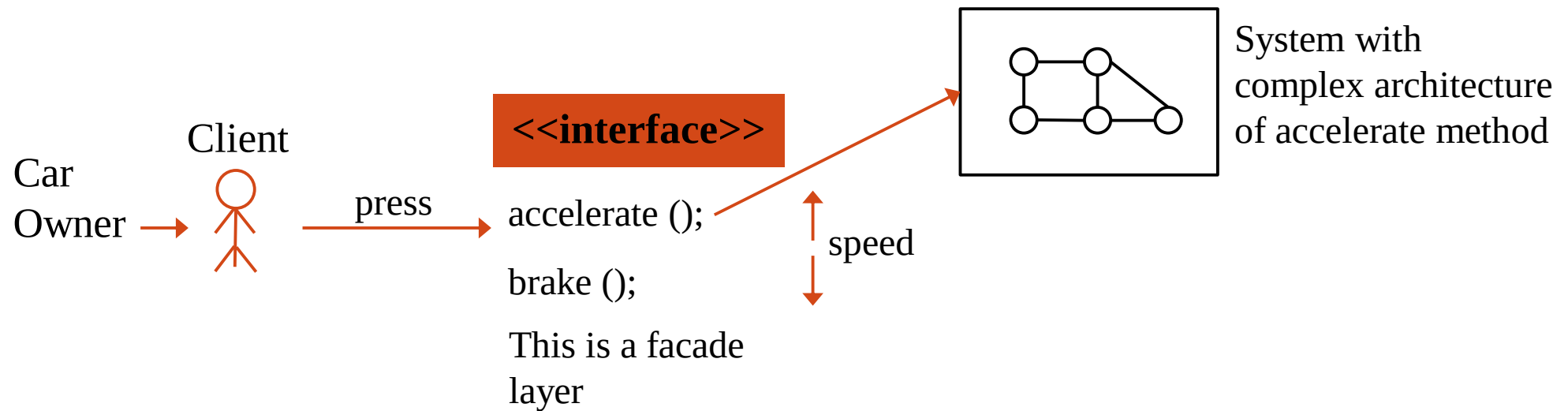
AKM Iqtidar Newaz [IqN]

Lecture - 16



FACADE DESIGN PATTERN

- ❑ Facade is a structural design pattern that provides a simplified interface to a library, a framework, or any other complex set of classes.
- ❑ When we have to hide the system complexity from the client we use facade design pattern.



EXAMPLE: SMARTHOME

```
#include <string>
#include <iostream>

using namespace std;

class Alarm
{
public:
    void alarmOn()
    {
        cout << "Alarm is on and house is secured" << endl;
    }
    void alarmOff()
    {
        cout << "Alarm is off and you can go into the house" << endl;
    }
};
```

main.cpp

```
class AirConditioner
{
public:
    void acOn()
    {
        cout << "Ac is on" << endl;
    }
    void acOff()
    {
        cout << "Ac is off" << endl;
    }
};

class TV
{
public:
    void tvOn()
    {
        cout << "Tv is on" << endl;
    }
    void tvOff()
    {
        cout << "Tv is off" << endl;
    }
};
```

main.cpp



EXAMPLE: SMARTHOME

```
class HouseFacade
{
private:
    Alarm alarm;
    AirConditioner ac;
    TV tv;

public:
    HouseFacade() {}

    void goToWork()
    {
        ac.acOff();
        tv.tvOff();
        alarm.alarmOn();
    }

    void comeHome()
    {
        alarm.alarmOff();
        ac.acOn();
        tv.tvOn();
    }
};
```

main.cpp

```
int main()
{
    HouseFacade hf;

    // Rather than calling 100 different on and off functions thanks to facade we only have 2 functions...
    cout << "Going to work:" << endl;
    hf.goToWork();
    cout << "\nJust got home:" << endl;
    hf.comeHome();
}
```

Output:

Going to work:

Ac is off

Tv is off

Alarm is on and house is secured

Just got home:

Alarm is off and you can go into the house

Ac is on

Tv is on

